



Introduction to

C Programming

Programs, languages, compilation, program structure,
tokens, data types and input / output statements

1 What is a Program?

A finite, ordered set of instructions that tells a computer how to solve a problem

A computer program is a collection of instructions, written in a programming language, that a computer executes in a definite sequence to transform input data into a required output.

Input

The raw data the program receives — typed by the user, read from a file

>

Process

The logic: calculations, comparisons, decisions and repetitions applied to that data by the CPU.

>

Output

The useful result — printed on screen, saved to a file, or sent to another device.

Key characteristics of a program

- Finiteness — it must terminate after a limited number of steps
- Definiteness — every instruction is precise and unambiguous
- Sequence — instructions execute in a defined order
- Effectiveness — each step is basic enough for the machine to perform

```
/* A program that adds two numbers */  
read a, b      <-- INPUT  
sum = a + b    <-- PROCESS  
print sum      <-- OUTPUT  
  
Program = algorithm + syntax
```

2

Algorithm, Program and Software

Three related terms that students often mix up

Algorithm

A step-by-step method to solve a problem, written in plain English or pseudocode. Language independent.

```
Add a and b, then display the result
```

Program

That algorithm translated into the syntax of a specific programming language so a machine can run it.

```
printf("%d", a + b);
```

Software

One or more programs bundled with data files, libraries and documentation, delivered to an end user.

```
A calculator application
```

Remember: The algorithm is the thinking. The program is the writing. The software is the product you ship.

3

What is a Programming Language?

A formal notation with its own vocabulary and grammar, used to communicate instructions to a computer

A programming language is a set of symbols, keywords and rules (syntax) together with a meaning for each construct (semantics), which lets a programmer express an algorithm in a form that can be translated into machine instructions.

Every language has three parts

Syntax

The grammatical rules. Where semicolons, braces and brackets must appear. Breaking syntax gives a compile-time error.

Semantics

The meaning behind a correct statement — what actually happens when it runs. Wrong semantics gives a logical error.

Vocabulary

The reserved words and symbols the language recognises, such as `int`, `if`, `while` and `return` in C.

Why do we need one?

- Computers understand only binary 0s and 1s — humans do not think that way
- A language is the bridge between human logic and machine hardware
- It makes programs readable, reusable and portable between machines

Same instruction, three levels:

```
Machine   : 10110000 01100001
Assembly  : MOV AL, 61h
C         : int x = 97;
```

4

Types of Programming Languages

Languages are classified by how close they sit to the hardware

1

Machine Language

First generation (1GL)

Written purely in binary 0s and 1s. Directly executed by the CPU with no translation. Extremely fast but almost impossible to write, read or debug, and completely machine dependent.

2

Assembly Language

Second generation (2GL)

Replaces binary with short mnemonics such as MOV, ADD and JMP. Needs an assembler to translate it. Easier than binary but still tied to one processor family.

3

High-Level Language

Third generation (3GL)

Uses English-like keywords and mathematical notation — C, C++, Java, Python. Needs a compiler or interpreter. Portable, readable and far faster to develop in.

Moving down the list: harder for the machine, easier for the human. C sits at the high-level end but keeps low-level power.

The C Programming Language

A general-purpose, structured, middle-level language that still powers the modern world

C was developed by Dennis Ritchie at AT&T Bell Laboratories in 1972 to rewrite the UNIX operating system. It is called a middle-level language because it offers high-level structure together with low-level memory access.

How C evolved

- **1960** ALGOL 60 — the common ancestor of structured languages
- **1967** BCPL by Martin Richards — typeless system programming
- **1970** B by Ken Thompson — a stripped-down BCPL for UNIX
- **1972** C by Dennis Ritchie — adds data types and structures
- **1989** ANSI C (C89) — the first official standard, then ISO C90
- **1999 →** C99, C11, C17 and C23 add modern features and safety

1972

Year created

Dennis Ritchie

Creator

Bell Labs

Birthplace

C17 / C23

Current standards

Where C is used today

- Operating systems and kernels
- Embedded systems, microcontrollers
- Compilers, interpreters, assemblers
- Device drivers and firmware
- Databases — MySQL, SQLite
- Graphics engines and games
- IoT and real-time systems

6

Features of C

The qualities that keep a fifty-year-old language in constant use

1 Simple and Efficient

A small core of just 32 keywords and a clean syntax, yet powerful enough to write an entire operating system.

3 Structured

Programs are broken into functions and blocks, which makes code easier to write, test, debug and reuse.

5 Pointers

Direct access to memory addresses enables dynamic memory, efficient arrays and hardware-level programming.

7 Extensible

You can add your own functions to the library and reuse them across many projects.

2 Portable

Code written on one machine runs on another with little or no change, so long as it avoids machine-specific tricks.

4 Rich Library

A large standard library — `stdio.h`, `math.h`, `string.h` and more — supplies ready-made functions.

6 Fast Execution

Compiled straight to native machine code, so C programs run close to assembly speed.

8 Recursive & Modular

Functions may call themselves, and programs can be split across several source files.

Compiler vs Interpreter

Two different ways of translating source code — C uses a compiler

Compiler

- Translates the entire program at once before execution
- Produces a separate executable file you can run repeatedly
- Reports all errors together after scanning the whole program
- Execution is fast because translation happens only once
- Needs more memory during the translation step
- Used by C, C++, Java (to bytecode), Go and Rust

Interpreter

- Translates and executes the program one line at a time
- Produces no separate executable — it runs from the source
- Stops at the first error it meets and reports only that one
- Execution is slower because translation repeats every run
- Needs less memory, since only one line is processed at a time
- Used by Python, JavaScript, Ruby and PHP

Which is better? Neither. A compiler suits finished software that must run fast; an interpreter suits learning, scripting and rapid experimentation where quick feedback matters more than raw speed.

8

Structure of a C Program

Every C program you write in this lab looks like this. Type it exactly as shown.

```
#include <stdio.h>

int main()
{
    printf("Hello, World!");
    return 0;
}
```

```
Save it as hello.c, then in the terminal:
$ gcc hello.c -o hello
$ ./hello
Hello, World!
```

1 `#include <stdio.h>`

Brings in the ready-made printing tools. Every program starts with this line.

2 `int main()`

The starting point. The computer begins running your program here.

3 `{ and }`

The braces hold the body. Everything the program does goes between them.

4 `return 0;`

Tells the computer the program finished properly. Always the last line.

For now: do not worry about why each line is there. Copy the shape, change only what is inside `main()`, and it will work.

9

Rules Every C Program Must Follow

Small habits that prevent most beginner errors

Case sensitivity

C treats uppercase and lowercase as different. `main`, `Main` and `MAIN` are three separate names.

Braces define blocks

Each opening `{` needs a matching `}`. Functions, loops and conditions all use them.

Exactly one `main()`

Execution always begins at `main()`. A program without it will link but never run.

Free-form layout

White space and new lines are ignored, but indentation makes code readable for humans.

Statement terminator

Every statement ends with a semicolon. A missing `;` is the single most common compile error.

Declare before use

A variable must be declared with a data type before any statement uses it.

Keywords are reserved

The 32 keywords have fixed meanings and can never be used as variable names.

Comments are for humans

Use `/* */` or `//` to explain intent. They cost nothing at run time.

The smallest individual units of a C program — the compiler reads code as a stream of tokens

A token is the smallest meaningful element the compiler can recognise. When you write `sum = a + b;` the compiler sees six tokens: the identifier `sum`, the operator `=`, the identifier `a`, the operator `+`, the identifier `b`, and the special symbol `;`

1 Keywords

32 reserved words with a fixed meaning, always lowercase

```
int, float, if, while, return
```

2 Identifiers

Names the programmer gives to variables, functions and arrays

```
sum, total_marks, name
```

3 Constants

Fixed values that never change during execution

```
25, 3.14, 'A', 0x1F
```

4 Strings

A sequence of characters enclosed in double quotes, ending with `\0`

```
"Hello", "C Language"
```

5 Operators

Symbols that perform arithmetic, logical or relational work

```
+, -, *, /, %, ==, &&
```

6 Special Symbols

Punctuation that structures the program

```
{ } [ ] ( ) ; , #
```

11

Keywords and Identifiers

Names the language reserves, and names you are free to invent

The 32 keywords of standard C

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

About keywords

- Always written in lowercase
- Meaning is fixed by the compiler
- Cannot be used as variable names
- C99 and C11 add a few more, such as `_Bool` and `inline`

Identifiers — the names you choose

Rules for naming

- May contain letters, digits and the underscore only
- Must begin with a letter or an underscore, never a digit
- No blank spaces or special symbols such as `@`, `#` or `-`
- Case sensitive — `Total` and `total` are different
- Cannot be a keyword

Valid vs invalid

<code>total</code>	valid
<code>roll_no</code>	valid
<code>_count</code>	valid
<code>Marks2</code>	valid
<code>2marks</code>	starts with a digit
<code>total marks</code>	contains a space
<code>net-pay</code>	hyphen not allowed
<code>float</code>	reserved keyword

Constants, Strings, Operators and Symbols

The remaining four token categories in detail

Constants

A constant is a value that cannot be altered while the program runs. C recognises four kinds:

- Integer constants — 25, -7, 0x1F (hex), 017 (octal)
- Real constants — 3.14, -0.5, 2.5e3
- Character constants — 'A', '9', '\n' in single quotes
- String constants — "Hello" in double quotes

Strings

A string is an array of characters ending with the null character \0, which the compiler appends automatically. The word "C" occupies two bytes: the letter and the terminator.

```
char name[6] = "HELLO";
H  E  L  L  O  \0
```

Operators

Arithmetic

+ - * / %

Relational

< > <= >= == !=

Logical

&& || !

Assignment

= += -= *= /=

Increment

++ --

Bitwise / Other

& | ^ ~ << >> ?:

Special Symbols

{ }

Open and close a block of statements

()

Function calls and grouping in expressions

[]

Array subscripts — marks[10]

;

Terminates every statement

,

Separates variables in a declaration or list

#

Marks a preprocessor directive

13

Data Types

A data type tells the compiler how much memory to reserve and how to interpret the bits stored there

When you write `int age;` the compiler reserves 4 bytes and records that those bytes hold a whole number. The data type therefore decides the size, the range of values, and the operations allowed on the variable.

Primary

Also called basic or built-in types, supplied by the language itself.

```
int    float
char   double
void
```

Derived

Built from the primary types by combining or referencing them.

```
array
pointer
function
```

User-defined

Types the programmer creates to model real-world data.

```
struct
union
enum, typedef
```

Type qualifiers and modifiers

short / long

Change how many bytes the type occupies

signed / unsigned

Decide whether negative values are allowed

const

Makes the variable read-only after initialisation

static / extern

Control lifetime and visibility across files

Primary Data Types — Size and Range

Sizes shown are for a typical 32/64-bit compiler; always confirm with sizeof()

Data Type	Size	Range	Specifier	Example
<code>char</code>	1 byte	-128 to 127	<code>%c</code>	<code>'A'</code>
<code>unsigned char</code>	1 byte	0 to 255	<code>%c</code>	<code>'z'</code>
<code>short int</code>	2 bytes	-32,768 to 32,767	<code>%hd</code>	1500
<code>int</code>	4 bytes	-2,147,483,648 to 2,147,483,647	<code>%d</code>	25
<code>unsigned int</code>	4 bytes	0 to 4,294,967,295	<code>%u</code>	4000
<code>long int</code>	4 / 8 bytes	-2,147,483,648 to 2,147,483,647	<code>%ld</code>	123456L
<code>long long int</code>	8 bytes	$\pm 9.22 \times 10^{18}$	<code>%lld</code>	900000000
<code>float</code>	4 bytes	1.2E-38 to 3.4E+38 (6 digits)	<code>%f</code>	3.14f
<code>double</code>	8 bytes	2.3E-308 to 1.7E+308 (15 digits)	<code>%lf</code>	3.141592
<code>long double</code>	10 / 16 bytes	3.4E-4932 to 1.1E+4932	<code>%Lf</code>	3.14159L
<code>void</code>	0 bytes	No value — used for empty return types	—	<code>void f()</code>

Check any size on your own machine with: `printf("%zu", sizeof(int));`

Format Specifiers

Placeholders beginning with % that tell printf and scanf how to interpret a value

A format specifier is a code inside a format string that is replaced by the value of a variable. It must match the variable's data type — printing a float with %d gives garbage, not an error.

%d

Signed decimal integer

```
printf("%d", 25);
```

%u

Unsigned decimal integer

```
printf("%u", 4000u);
```

%s

A string of characters

```
printf("%s", name);
```

%lf

Double precision value

```
scanf("%lf", &d);
```

%o

Octal (base 8) integer

```
printf("%o", 64);
```

%p

Memory address of a pointer

```
printf("%p", &a);
```

%i

Integer; in scanf detects base automatically

```
scanf("%i", &n);
```

%c

A single character

```
printf("%c", 'A');
```

%f

Float in decimal notation

```
printf("%f", 3.14);
```

%e / %E

Scientific notation

```
printf("%e", 12345.0);
```

%x / %X

Hexadecimal (base 16) integer

```
printf("%x", 255);
```

%%

Prints a literal percent sign

```
printf("50%%");
```

16 Output Statement — printf()

The formatted output function declared in stdio.h

Syntax: `printf("control string", variable1, variable2, ...);`

What the control string may contain

1 Ordinary text

Printed exactly as written — `printf("Hello");` displays Hello

2 Format specifiers

Replaced by the value of the matching argument, in order

3 Escape sequences

Special characters such as `\n` for a new line or `\t` for a tab

```
int age = 20;
float cgpa = 8.567;
char grade = 'A';

printf("Age    : %d\n", age);
printf("CGPA   : %.2f\n", cgpa);
printf("Grade  : %c\n", grade);
```

Common escape sequences

`\n`

new line

`\t`

tab

`\\`

backslash

`\"`

quote

`\0`

null

`\a`

beep

Output

```
Age    : 20
CGPA   : 8.57
Grade  : A
```

Input Statement — scanf()

The formatted input function that reads values from the keyboard

Syntax: `scanf("control string", &variable1, &variable2, ...);`

Why the & symbol?

& is the address-of operator. scanf must know where in memory to store the value it reads, so you pass the address of the variable rather than its value. Forgetting & is the most frequent scanf mistake and usually crashes the program.

Points to remember

- String variables need no & — an array name is already an address
- Use %lf for double in scanf, even though %f works in printf
- scanf stops reading a string at the first white space
- It returns the number of items successfully read
- Multiple values can be read in one call: `scanf("%d %d", &a, &b);`

```
int roll;
float marks;
char name[30];

printf("Enter roll no: ");
scanf("%d", &roll);

printf("Enter name: ");
scanf("%s", name);
           ^ no & needed
```

Unformatted I/O functions

<code>getchar()</code> / <code>putchar()</code>	read and write one character
<code>gets()</code> / <code>puts()</code>	read and write a whole line
<code>getch()</code> / <code>getche()</code>	read a key without pressing Enter